Mini Job Portal & Resume Matcher

High-Yield Interview Revision Sheet & Technical Architecture Deep-Dive

TECHNICAL OVERVIEW

Full-Stack Microservice Pipeline

1. Executive Summary & 30-Second Interview Pitch

> **The 30-Second Pitch (Memorize this!):**
> *"I developed a decoupled, multi-service Mini Job Portal where candidates apply for jobs and recruiters see applicants instantly ranked by an automated resume-match percentage. Instead of forcing all workloads into a single runtime, I assigned each technology its natural strength: **React** delivers a responsive candidate and recruiter SPA, **PHP** acts as an isolated, secure file ingestion microservice validating real MIME types and size constraints, **Python** extracts resume text and performs keyword scoring, **Node/Express** coordinates the REST API workflow, and **MongoDB** stores flexible job schemas and ranked applications."*

2. Architectural Matrix: Why Each Tech Fits Naturally

Technology	Port	Assigned Role	Why It Fits Naturally (Interview Defense)
React.js	`3000`	Candidate & Recruiter SPA	Instant job filtering by skill pills, two-phase apply modal with live microservice progress trackers, and real-time applicant leaderboard.
PHP	`8000`	Resume Ingestion (`upload.php`) + Feedback	PHP's native C-level engine handles multipart uploads out of the box without bloating the Node event loop. Independent 5-layer security wall.
Node.js + Express	`5000`	Core REST API & Orchestrator	Ideal I/O coordinator: handles Jobs CRUD, triggers Python matching upon application submission, serves static uploaded files, and persists to DB.
Python	`5001`	Microservice (`/match`)	Best-in-class text extraction ecosystem (`pypdf`) and deterministic whole-word regex matching engine for accurate scoring.
MongoDB	`27017`	Document Database	JSON-native document store: naturally accommodates dynamic `skills[]` arrays, match percentages, and supports indexed sorting.

3. End-to-End Execution Flow (Data Lifecycle)

1. **Candidate Application:** Candidate selects a job opening in React, fills in contact details, and selects a resume file (PDF, DOCX, TXT).
2. **Direct Ingestion to PHP:** React sends a `multipart/form-data` POST to `http://127.0.0.1:8000/upload.php`. PHP validates size (≤5MB), extension whitelist, and inspects real binary MIME magic bytes via `finfo_file`.
3. **Safe Storage & Path Return:** PHP generates a cryptographically random unique filename (`resume_{time}_{hex}.pdf`), calls `move_uploaded_file()`, and returns a JSON payload with `relativePath` and `fileName`.
4. **Application Submission to Express:** React passes the candidate payload along with the PHP-verified `resumePath` to Express at `POST /api/applications`.

5 **Python Keyword Match Execution:** Express fetches required skills from the Job record and dispatches an HTTP POST to Python at `http://127.0.0.1:5001/match`. Python extracts text with `pypdf` and calculates: $$\text{Match Percentage} = \left(\frac{\text{matched_skills}}{\text{required_skills}}\right) \times 100$$

6 **Database Persistence & Recruiter Dashboard:** Express saves the candidate with `matchPercentage` and `matchedSkills` into MongoDB. Recruiter dashboard fetches applicants sorted descending: `.sort({ matchPercentage: -1 })`.

💡 **Key Takeaway for Interviews**

PHP and Python operate as **stateless worker microservices**. Express handles the database and workflow coordination. This eliminates the need for PHP-to-MongoDB drivers or Python ORM overhead!

4. Deep-Dive: PHP File Upload & Security Architecture

File upload vulnerabilities are one of the most common security questions in interviews. Here is the 5-layer defense implemented in `upload.php`:

Layer 1: Denial-of-Service (DoS) File Size Limit

DDOS DEFENSE

Files are capped at 5MB (`$file['size'] > 5 * 1024 * 1024`). Checked before copying to disk to prevent disk exhaustion attacks.

Layer 2: Strict Extension Whitelisting

INPUT SANITIZATION

Uses `pathinfo($file['name'], PATHINFO_EXTENSION)` against strict whitelist: `['pdf', 'doc', 'docx', 'txt']`. Rejects any unapproved extension immediately with HTTP 422.

Layer 3: Magic Bytes / Binary MIME Inspection

CRITICAL DEFENSE

Never trust `$_FILES['resume']['type']`! The browser sends this header and any attacker with Postman or Burp Suite can fake it (e.g. sending a PHP web shell with `Content-Type: application/pdf`).
Solution: We inspect the file's raw magic bytes on disk using `finfo_open(FILEINFO_MIME_TYPE)` and `finfo_file()`. Only true binary MIME types (`application/pdf`, `application/msword`, etc.) pass.

Layer 4: Cryptographically Random Filename Generation

PATH TRAVERSAL DEFENSE

Attackers often submit filenames with path traversal strings like `../../var/www/shell.php` or filenames designed to overwrite critical files. We discard the original filename and generate:

```
$safeFileName = 'resume_' . time() . '_' . bin2hex(random_bytes(8)) . '.' . $extension;
```

Ensures zero collisions and complete immunity against Directory Traversal attacks.

Layer 5: Server-Level Execution Prevention via .htaccess

DEFENSE-IN-DEPTH

Inside `php-service/uploads/.htaccess`, script execution is disabled completely:

```
<FilesMatch "\.(php|php5|phtml|sh|py|cgi)$">
    Order Deny,Allow
    Deny from all
</FilesMatch>
php_flag engine off
```

Even if an attacker somehow uploaded an executable script, the web server refuses to execute it.

5. Deep-Dive: Python Matching Engine & Trade-offs

A. PDF Text Extraction Mechanics

The Python microservice (`app.py`) utilizes `pypdf` (or `PyPDF2`) to read binary PDF content page by page and extract plain character streams. Extracted text is normalized to lowercase and stripped of unnecessary symbols.

> ⚠️ **The Scanned PDF Edge Case (Crucial Interview Discussion Point)**
>
> **Problem:** When a candidate uploads a *scanned PDF* (a photograph/scan of a paper resume without an embedded digital text layer), `pypdf` returns empty string `""`.
>
> **Solution:** Our service checks `len(extracted_text.strip()) < 10`. If empty, it flags `isScannedPdf = True` with a descriptive warning: "*Scanned PDF detected without OCR text layer*".
>
> **Trade-off:** Adding Tesseract OCR requires heavy native C++ binaries (`tesseract-ocr` , `leptonica`) and takes 3–8 seconds per page. For a lightweight mini portal, documenting this architectural limitation and handling it gracefully is the right engineering decision.

B. Robust Regex Matching for Special Tech Keywords

Standard regex word boundary `\b` fails on technical terms with non-alphanumeric symbols (e.g. `\bC++\b` fails because `+` is not a word character). Our Python matcher dynamically selects the appropriate pattern:

```
# If skill contains +, #, ., or - (e.g. C++, C#, .NET, Node.js)
pattern = r"(?:^|[\s,;:(/])" + re.escape(skill.lower()) + r"(?:$|[\s,;:)/])"
# Standard alphanumeric keywords (e.g. React, MongoDB, Docker)
pattern = r"\b" + re.escape(skill.lower()) + r"\b"
```

C. Why Deterministic Keyword Matching vs "AI/ML"?

Calling this project "AI-powered" in an interview is a common trap. It is a **deterministic keyword matching engine**. Deterministic matching has zero hallucinations, executes in under 15ms, requires no costly GPU infrastructure, and produces a mathematically verifiable score that recruiters can audit.

6. Deep-Dive: Express Backend & Database Schema

MongoDB Schema Design

Collection	Field	Type	Purpose / Indexing
jobs	`title`, `company`	String	Indexed for keyword searches
	`location`, `salary`	String	Display metadata
	`skills`	[String]	Array of required tech stacks used for Python match input
	`createdAt`	Date	Default `Date.now` (sort newest first)
applications	`jobId`	ObjectId	Foreign reference to `jobs._id`
	`candidateName`, `email`	String	Candidate contact details
	`resumePath`, `resumeFileName`	String	Relative path to file stored by PHP upload service
	`matchPercentage`	Number	**Indexed with descending order:** `{ matchPercentage: -1 }`
	`status`	Enum String	`'applied'` | `'shortlisted'` | `'rejected'`

Database-Level Sorting Optimization

In `applicationRoutes.js`, recruiter queries are sorted at the database engine level:

```
const applicants = await Application.find({ jobId }).sort({ matchPercentage: -1 });
```

Interview Point: Sorting in MongoDB using an index is $O(\log N)$, whereas fetching unsorted records and sorting inside React or Node memory is $O(N \log N)$ and degrades client performance when candidate volume grows.

7. Top 15 Technical Interview Questions & High-Impact Answers

Q1: Why did you use PHP for file upload instead of Node's Multer? — Architecture

"PHP's C-level engine has native, battle-tested streaming multipart upload handling. In our microservice architecture, PHP serves as an isolated file ingestion sandbox. Offloading large file uploads and MIME type binary checks prevents Node's single-threaded event loop from being bogged down by I/O file stream processing."

Q2: Why not connect PHP directly to MongoDB? — Microservice Design

"Connecting PHP to MongoDB would require compiling the PECL MongoDB extension and installing Composer libraries, adding unnecessary runtime coupling. By keeping PHP completely stateless—responsible only for validating and writing the file—Express handles all database operations cleanly."

Q3: What does move_uploaded_file() do and why not use copy() or rename()? — Security

"`move_uploaded_file()` internally calls `is_uploaded_file()` to verify that the file was genuinely received through an HTTP POST upload request. If an attacker tricks a script into moving an arbitrary system file (like `/etc/passwd`), `copy()` would proceed, but `move_uploaded_file()` will strictly fail."

Q4: Why is checking $_FILES['resume']['type'] dangerous?

Security

"*`$_FILES['resume']['type']` is purely client-supplied from the HTTP `Content-Type` request header. A malicious user can upload a PHP script named `exploit.php` while setting the header to `application/pdf`. Instead, we use PHP's `finfo_file()` to read magic bytes directly from the binary file stream.*"

Q5: How does your Python service handle skills like 'C' vs 'C++' or 'Java' vs 'JavaScript'?

Python & Regex

"*Naive string containment (`skill in text`) would cause 'Java' to match on 'JavaScript', and standard word boundary `\b` fails on symbols like `+`. Our Python matcher detects special characters and uses custom character class boundaries `(?:^|[\s,;:(/])(?:$|[\s,;:)/])` to guarantee exact token matching.*"

Q6: What happens if a candidate uploads an image or scanned PDF?

Edge Cases

"*Standard PDF extractors extract digital text streams. Scanned PDFs are pixel images. Our Python matcher checks if extracted text length is near zero, flags `isScannedPdf = True`, and returns a graceful diagnostic warning. In a production pipeline, this would trigger an asynchronous OCR worker queue (using Tesseract/Celery).*"

Q7: Why did you intentionally skip Authentication / Login in this project?

Scope & Tradeoffs

"*I prioritized engineering depth on the core business differentiator: multi-runtime file validation, secure stream ingestion, and automated keyword scoring pipeline. Standard JWT auth is a generic boilerplate feature that would dilute time away from perfecting the multi-service architecture.*"

Q8: How does the Recruiter Dashboard rank applicants in real-time?

Full Stack

"When Express receives the application, it stores the calculated `matchPercentage` directly in the MongoDB document. When the recruiter selects an opening, Express queries `Application.find({ jobId }).sort({ matchPercentage: -1 })`. React renders them with color-coded score badges (Green ≥70%, Yellow ≥40%, Red <40%), showing matched and missing skill breakdown."

Q9: How are uploaded files accessed by the recruiter?

Express I/O

"Express serves the PHP uploads folder as a static resource using `app.use('/uploads', express.static(phpUploadsDir))`. The recruiter dashboard provides a direct 'View File' link to inspect the original PDF safely."

Q10: What if the Python matching microservice goes down?

Resilience

"In `applicationRoutes.js`, the axios call to the Python service is wrapped in a `try/catch` with a 3.5-second timeout. If Python is unreachable, Express falls back to an internal keyword matching algorithm so the candidate's application is never lost."

Q11: How do you handle CORS across these three distinct ports?

Web Tech

"The frontend runs on port 3000, PHP on 8000, Node on 5000, and Python on 5001. Both PHP and Flask explicitly handle HTTP OPTIONS preflight requests and supply `Access-Control-Allow-Origin: *` and allowed headers, enabling cross-origin communication."

Q12: How would you scale this architecture to handle 50,000 resumes a day?

System Design

*"Instead of synchronous HTTP requests from Express to Python, I would introduce a message broker like **RabbitMQ** or **Redis BullMQ**. Express would publish a job with the resume path and immediately return `status: 'processing'`. A pool of Python worker processes would consume jobs off the queue, calculate match scores, and update MongoDB asynchronously via WebSockets."*

Q13: How does your React UI prevent duplicate applications or bad file uploads?

Frontend Validation

"React conducts immediate client-side checks on file extension (`.pdf, .doc, .docx, .txt`) and size (≤5MB) before dispatching the network request. During submission, button states are disabled and a live stepper indicator informs the user of upload and scoring progress."

Q14: Where does the contact/feedback form fit into this system?

PHP Handler

"Alongside `upload.php`, we implemented `feedback.php` to demonstrate standard PHP form processing: sanitizing text with `strip_tags()`, validating emails with `FILTER_VALIDATE_EMAIL`, and logging applicant/recruiter feedback."

Q15: What is the single biggest lesson you learned from building this project?

Engineering Maturity

"Polyglot architectures must be justified by technical advantages rather than novelty. PHP excels at secure multipart file ingestion, Python excels at text processing, Node excels at asynchronous API orchestration, and React provides clean state management. Keeping each service decoupled and stateless created a clean, maintainable system."

8. Quick Recall Checklist (Check 10 Minutes Before Interview)

Area	Recall Bullet Points
PHP Upload Security	`5MB limit` → `Whitelist ext` → `finfo_file() magic bytes` → `bin2hex(random_bytes(8)) unique name` → `move_uploaded_file()` → `.htaccess script execution deny`.
Python Matching	`pypdf` page reader → whole-word regex pattern → $(matched / required) \times 100$ → Scanned PDF warning. (Say: "*Keyword matching, not ML*").
Node + Express	Orchestrator between React, PHP, Python & Mongo → Static file serving for `/uploads` → Resilient fallback matcher.
MongoDB	`jobs` schema has `skills: [String]` → `applications` has `matchPercentage` → Indexed descending sort `{ matchPercentage: -1 }`.
Ports Quick Reference	React: `:3000` | PHP: `:8000` | Express: `:5000` | Python: `:5001` | Mongo: `:27017`.